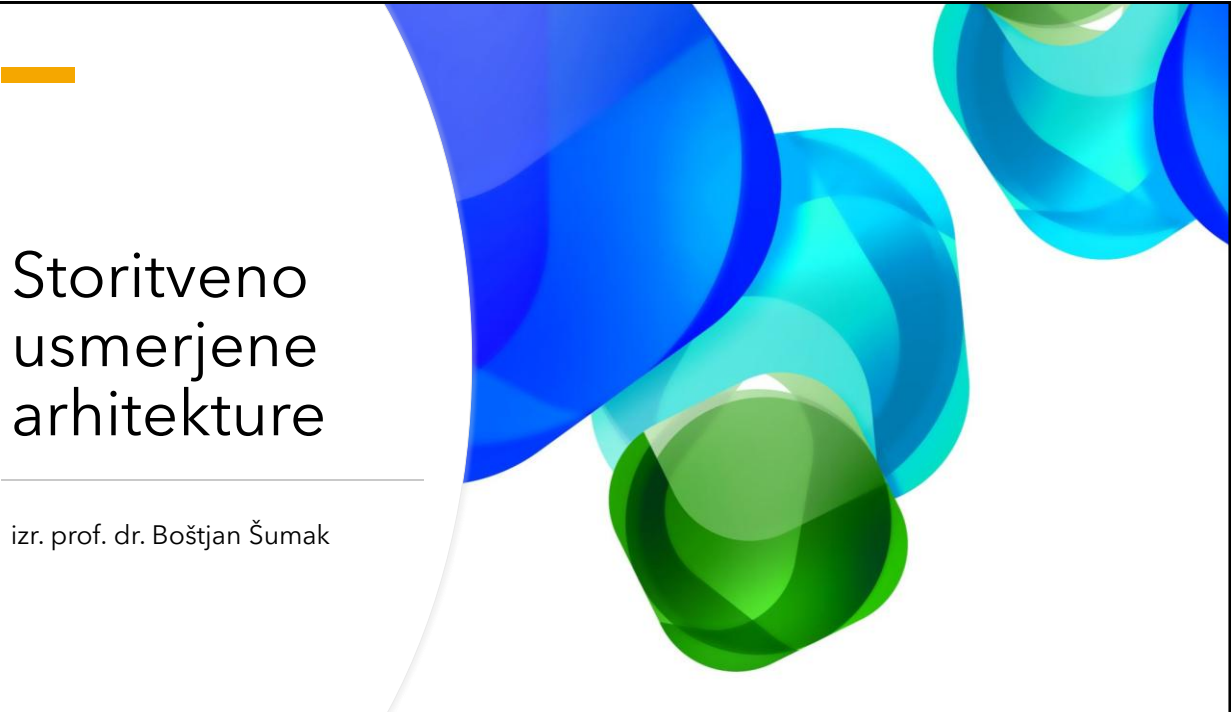




Storitveno usmerjene arhitekture

izr. prof. dr. Boštjan Šumak

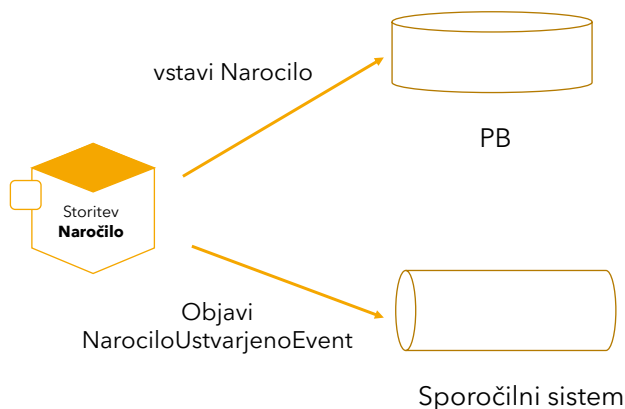
1

Nazadnje

- Arhitekturni vzorci v mikrostoritvenih rešitvah
 - Razdelitev sistema na mikrostoritve
 - Nameščanje storitev
 - Komunikacijski vzorci
 - API gateway
 - Med-procesna komunikacija
 - Obravnava delnih okvar
 - Odkrivanje storitev
 - Registracija storitev
 - Upravljanje porazdeljenih podatkov

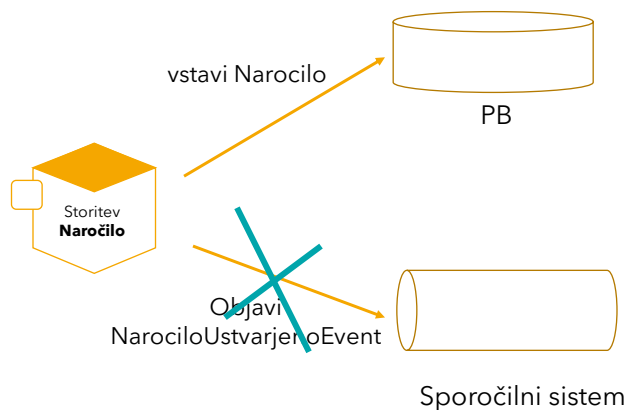
2

Kako posodobiti podatke in objaviti dogodek tako, da je oboje izvedeno transakcijsko (vse ali nič)?



3

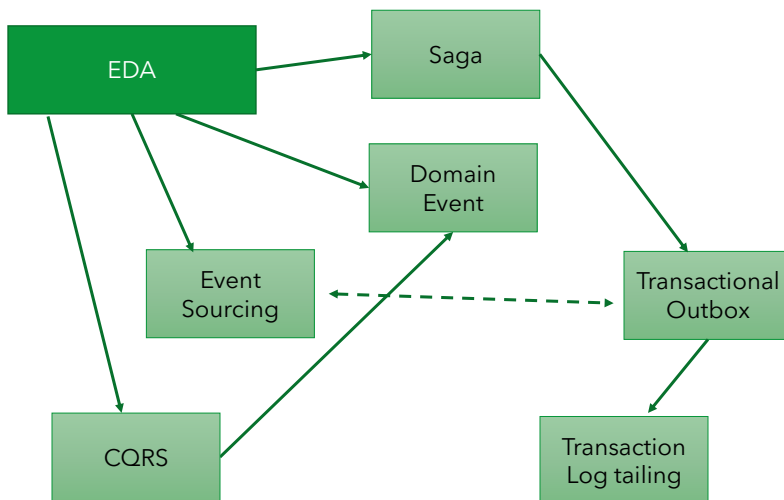
Kako posodobiti podatke in objaviti dogodek tako, da je oboje izvedeno transakcijsko (vse ali nič)?



Lahko se zgodi, da v PB uspemo zapisati novo naročilo, a ne uspemo objaviti dogodka v sporočilnem sistemu.

4

Zanesljiva objava dogodkov ob spremembi stanja



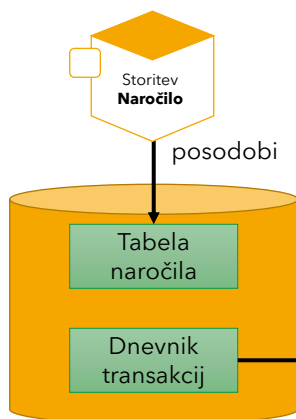
Zagotavljanje konsistentnosti

Prerejeno po: <https://microservices.io/i/MicroservicePatternLanguage.jpg>

5

Vzorec: Transaction log tailing

Storitev izvede spremembo podatkov v **podatkovni bazi** (INSERT/UPDATE/DELETE) kot del običajne transakcije.



Ločena komponenta na sistemski ravni **spremlja dnevnik transakcij** (transaction log) in iz njega zazna spremembe.

Ob zaznani spremembi komponenta ustvari in objavi ustrezen **dogodek** v **sporočilni sistem** (npr. Kafka, RabbitMQ).

Druge storitve se na dogodke naročijo in **posodobijo svoje lokalne kopije** ali sprožijo nadaljnjo obdelavo.

6

Transaction log tailing: primeri rešitev

- **LinkedIn Databus (CDC pipeline)** - <https://github.com/linkedin/databus>
 - Porazdeljen sistem za zajem sprememb iz podatkovnih baz z **rudarjenjem transakcijskih/commit dnevnikov** (log mining).
- **MongoDB: oplog + Change Streams** - <https://www.mongodb.com/docs/manual/core/replica-set-oplog>
 - MongoDB replike uporabljajo **oplog (operation log)** za beleženje sprememb.
 - V praksi se pogosto uporablja **Change Streams**, ki omogočajo spremljanje sprememb v realnem času brez neposrednega branja oploga
- **AWS DynamoDB Streams** - <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Streams.html>
 - Zajame **časovno urejeno zaporedje sprememb na nivoju elementov (item-level)** v tabeli in jih hrani **do 24 ur**
 - uporabniki berejo tok sprememb skoraj v realnem času
- **Debezium (na Kafka Connect)** - <https://debezium.io/documentation/reference/stable/architecture.html>
 - Platforma za CDC konektorje
 - za **MySQL bere binlog**, za **PostgreSQL bere logical replication stream** in spremembe objavlja v teme (topics) v Kafki.
- **AWS Database Migration Service (DMS) - CDC / ongoing replication** - <https://docs.aws.amazon.com/dms/>
 - Pri "ongoing replication" DMS zajema spremembe prek **podatkovnih dnevnikov** z uporabo **native API-jev** podatkovnih pogonov.
- **Oracle GoldenGate (log-based capture)** - <https://docs.oracle.com/en/middleware/goldengate/core/index.html>
 - Primer klasične rešitve za log-based CDC: za Oracle zajema iz **redo logov**, za MySQL iz **binary logov**.

7

Transaction log tailing: prednosti in slabosti

Prednosti:

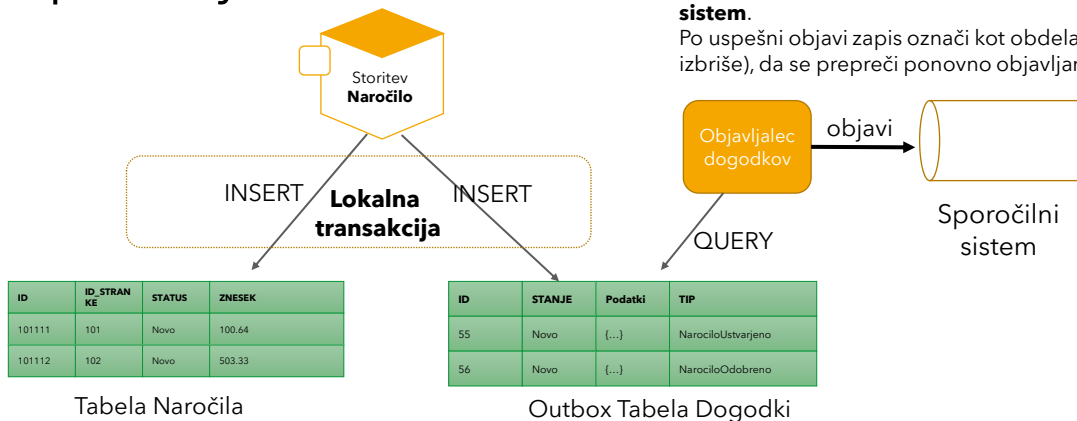
- **Brez sprememb v poslovni kodi:** spremembe zajema ločena CDC komponenta, storitve ostanejo preproste.
- **Točen zajem sprememb podatkov:** zajem temelji na transakcijskem dnevniku, zato praviloma ne "izpusti" commit-anih sprememb.
- **Manjša vezanost na klice med storitvami:** dogodki nastajajo iz sprememb v podatkih, porabniki se nanje naročijo.
- **Uporabno za integracije in replikacije:** dobra podlaga za sinhronizacijo projekcij (read modelov) in analitiko.

Slabosti:

- **Dogodki so na podatkovnem nivoju:** pogosto dobimo "row/item change events"; za poslovne dogodke je potrebna dodatna interpretacija (povratni inženiring domenskih dogodkov).
- **Odvisnost od podatkovnega pogona in njegovih mehanizmov:** različne baze imajo različne dnevnike, pravila in omejitve → več vezanosti na SUPB in orodja.
- **Kompleksnost semantike dostave:** vrstni red, duplikati, "at-least-once" obnašanje in idempotentnost morajo biti rešeni na strani porabnikov.
- **Operativna zahtevnost:** konfiguracija konektorjev, nadzor zaostanka (lag), obnova po napakah, upravljanje shem/sprememb strukture podatkov.

8

Vzorec: Dogodki, ustvarjeni iz strani aplikacije



9

Dogodki, ustvarjeni iz strani aplikacije: prednosti in slabosti

Prednosti:

- **Visokonivojski (domenski) dogodki:** ker jih ustvari aplikacija, dogodki odražajo poslovni pomen (npr. *OrderPlaced*), ne le sprememb v vrsticah tabel.
- **Konsistentnost z lokalno transakcijo:** zapis poslovnih podatkov in zapis dogodka sta v isti transakciji → zmanjša tveganje "podatki so shranjeni, dogodek pa manjka" (ali obratno).

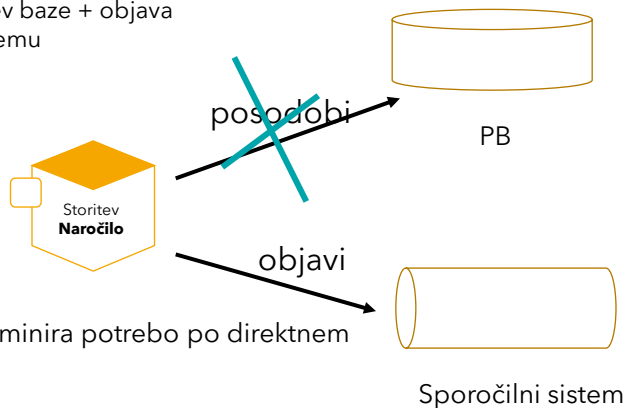
Slabosti:

- **Spremembe v aplikaciji:** potrebno je dodati outbox tabelo in logiko za zapis dogodkov.
- **Možni duplikati (at-least-once):** dogodek se lahko objavi večkrat (npr. ob ponovitvah/retryjih) → porabniki morajo biti **idempotentni**, dogodki naj imajo **unikaten ID**.
- **Omejitve po podatkovnih shrambah:** vzorec je najlažje izvesti v **SQL**; pri NoSQL je izvedba možna le, če shramba podpira ustrezne transakcije/atomske (ali primerljiv mehanizem).

10

Vzorec: Event-sourcing

Prej smo omenili izzive in rešitve, ki so povezane s transakcijsko izvedbo 2 osnovnih operacij, ko pride do spremembe: posodobitev baze + objava dogodka v sporočilnem sistemu



Vzorec Event-sourcing eliminira potrebo po direktnem posodabljanju PB.

**Storitev objavi samo dogodek o spremembi.
Stanje PB upravljamo preko objave dogodkov.**

11

Vzorec: Event-sourcing

- Je pristop, kjer stanje sistema ne shranjujemo kot "trenutno stanje", ampak kot **zaporedje dogodkov**.
- Namesto: "naročilo ima status POSLANO" shranimo:
 - "NaročiloUstvarjeno",
 - "PlačiloPotrjeno",
 - "NaročiloOdposlano"
 - ...
- Trenutno stanje dobimo tako, da dogodke ponovno predvajamo (replay) v pravilnem vrstnem redu.
- Dogodki so neizbrisljivi (append-only)
 - ne popravljamo zgodovine, ampak dodamo nov dogodek.

12

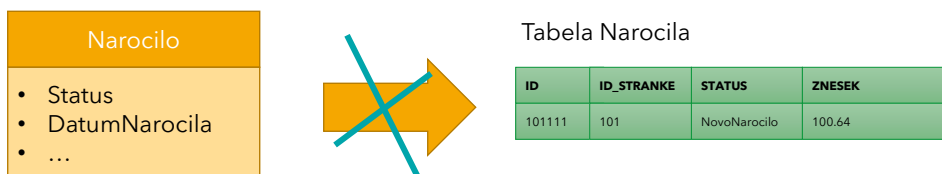
Vzorec: Event-sourcing

- Za vsako poslovno entiteto (ki je lahko agregacija več storitev) moramo:
 - Identificirati in specificirati domenske dogodke (spremembe stanja)
- Na primer:
 - **Košarica:** IzdelekDodan, IzdelekOdstranjen, NaročiloOddano, ...
 - **Naročilo:** NovoNarocilo, NaročiloPreklicano, NaročiloOdobreno, NaročiloZavrjeno, itd.

13

Vzorec: Event-sourcing

- V klasični obliki shranjevanja podatkov shranjujemo v tabelo stanje entitete, spreminjamo stanje entitete, itd.



Pri tehniki Event-sourcing ne zapisujemo podatkov v podatkovno tabelo oz. tabelo, ki je povezana s stanjem entitete!

Pri tehniki Event-sourcing v bazo (v tabelo dogodkov) zapišemo zaporedje dogodkov.

14

Event-sourcing – kako to izgleda v praksi

- **Primer: naročilo v spletni trgovini**

- Dogodki:

- *NaročiloUstvarjeno*(id, kupec, postavke)
- *PlačiloPotrjeno*(id, znesek)
- *NaročiloPreklicano*(id, razlog) ali
- *NaročiloOdposlano*(id, dostava)

- Iz dogodkov lahko določimo/izračunamo:

- trenutni **status** naročila,
- **skupno vrednost**,
- ali je naročilo veljavno za preklic,
- itd.

- **Ključna ideja:** baza vsebuje "kaj se je zgodilo", ne le "kako je zdaj".

15

Tehnika Event-sourcing primer zaporedja dogodkov v tabeli Events

Tabela Events

Aggregate id	Aggregate type	Event id	Event Type	Event Data
101	Narocilo	901	NarociloUstvarjeno	...
101	Narocilo	902	PlaciloPotrjeno	...
101	Narocilo	903	NarociloOdposlano	...

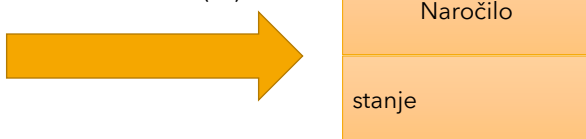
Pri tehniki Event-sourcing v bazi torej vodimo zapise o dogodkih nad poslovnimi entitetami sistema.

16

Ponovna izvedba dogodkov za poustvarjanje stanja

Dogodki:

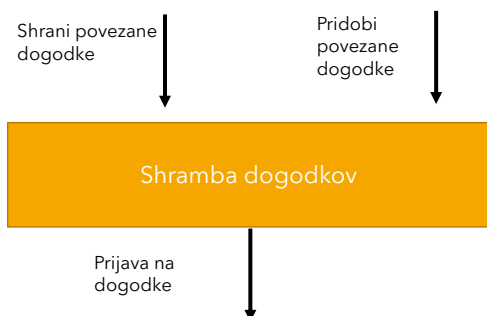
NovoNarocilo(...)
NarociloOdobreno(...)
NarociloPoslano(...)



17

Event-sourcing: Shramba dogodkov

- Event store mora hkrati:
 - trajno hraniti dogodke (kot baza),
 - omogočati distribucijo dogodkov porabnikom (kot pub/sub ali stream)



Možne rešitve:

- 1) Namenske "event-native" shrambe
 - **EventStoreDB** - dogodki so shranjeni kot tok dogodkov, z možnostjo streamanja in replay-a.
 - **Axon Server** - kombinacija event store + message delivery/router
- 2) Hibridne platforme (baza + broker)
 - **Eventuate** (Local) - kombinacija SQL baze + Kafka (persistenca + pub/sub).
- 3) "Lastne" implementacije

18

Event-sourcing: prednosti

Sledljivost in revizijska sled

- vedno vemo, *kdo je kaj naredil in kdaj*.

Ponovljivost

- lahko rekonstruiramo stanje sistema za poljuben trenutek v času (*time travel*).

Lažje povezovanje z drugimi storitvami

- dogodke lahko objavimo in drugi gradijo svoje projekcije.

Dobro se ujema z vzorcem **CQRS**

- zapisi ustvarjajo dogodke, branje uporablja optimizirane **projekcije** (read model).

19

Event-sourcing: Izzivi

- Zahteva, da aplikacijo/rešitev implementiramo od začetka
- Manj znan stil programiranja → učna krivulja
- Rekonstrukcija stanja iz mnogih dogodkov je lahko počasna
- Spremembe sheme dogodkov (verzioniranje) zahtevajo disciplino
- Težje "popravljanje" napak: zgodovine ne spreminjamo, dodajamo nove dogodke.
- Paziti je potrebno na duplikate dogodkov
 - idempotentni handlerji ali odkrivanje duplikatov

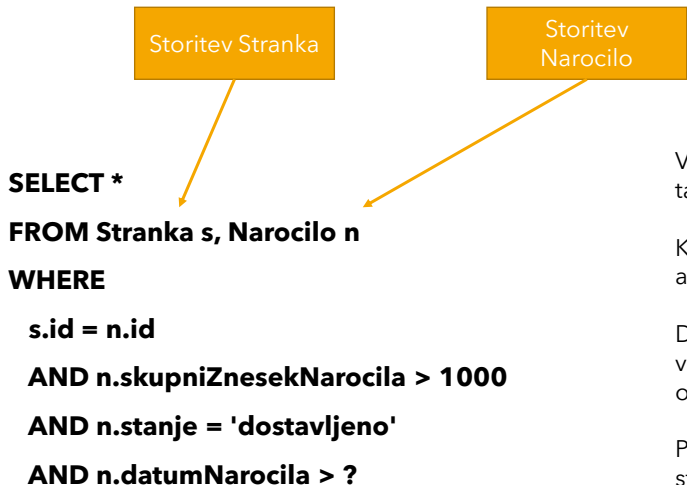
20

Vzorec „Event Sourcing“ in povpraševanje

- Vzorec „Event sourcing“ sicer rešuje problem posodabljanja podatkov, a hkrati vpelje nove izzive pri povpraševanju nad podatki
- Recimo, da moramo zagotoviti storitev za pridobivanje podatkov o strankah in njihovih zadnjih naročilih
 - Pri seznamu strank bi želeli samo seznam strank, ki
 - z naročili **v zadnjem tednu**, kjer je
 - **skupni znesek naročila > 1000**

21

Poizvedba: najdi stranke, ki so v zadnjem obdobju izvedle naročila >1000



```

SELECT *
FROM Stranka s, Narocilo n
WHERE
  s.id = n.id
  AND n.skupniZnesekNarocila > 1000
  AND n.stanje = 'dostavljeno'
  AND n.datumNarocila > ?
  
```

V primeru rešitve z 1 skupno bazo, tak query ni noben problem.

Kaj pa v primeru mikrostoritvene arhitekture?

Do storitev sočasno dostopajo veliko drugih storitev in odjemalcev

Podatki o stranki so v PB v okviru storitve Stranka, naročila pa v PB, ki jo upravlja storitev Naročila

22

Kako implementirati rešitev?

- V osnovi je potrebno združiti stranke in naročila na osnovi tujega ključa v tabeli naročilo

ampak

- predpostavimo, da shramba dogodkov omogoča pridobivanje podatkov samo na osnovi primarnega ključa entitete!
 - Iz shrambe dogodkov ni možno pridobiti podatke o naročilih stranke
- Rešitev: **Command Query Responsibility Segregation (CQRS)**

23

Kaj je CQRS?

- **CQRS (Command Query Responsibility Segregation)**

= ločitev odgovornosti za:

- **Command**: operacije, ki **spreminjajo stanje** (zapis)
- **Query**: operacije, ki **berejo stanje** (branje)
- Cilj: različne potrebe zapisa in branja rešujemo **ločeno**, namesto z enim samim modelom.

24

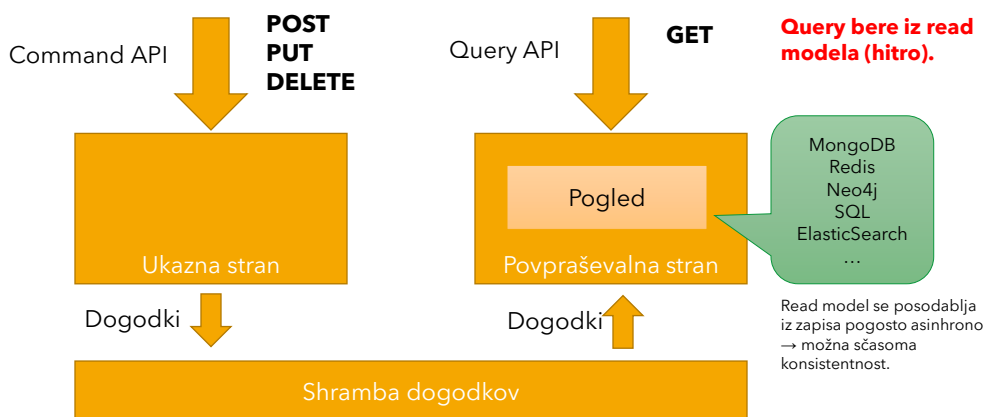
Zakaj CQRS?

- **Različne zahteve branja in pisanja** (npr. veliko branja, malo pisanja).
- **Optimizacija branja**: posebni "read modeli" po meri UI (manj joinov, hitrejše poizvedbe).
- **Neodvisno skaliranje**: posebej skaliramo del za branje in del za zapis.

25

Command Query Responsibility Segregation (CQRS)

- Na osnovi CQRS se sistem razdeli na 2 komponenti:
 - **Ukazna stran**, ki procesira ukaze HTTP PUT, POST in DELETE
 - **Povpraševalna stran**, ki procesira ukaze HTTP GET s povpraševanje nad podatki, ki so lahko v poljubni PB (MongoDB, Redis, ...)



26

CQRS + Event Sourcing - kako se dopolnjujeta

- **Event Sourcing** pogosto napaja CQRS:
 - Write model zapisuje **dogodke** v Event Store.
 - Read modeli nastajajo kot **projekcije** iz dogodkov.
- Prednost: **močna sledljivost + več "pogledov"** na iste podatke.
- CQRS lahko implementiramo **tudi brez** event sourcinga
 - npr. klasična relacijska baza za write, projekcije za read

27

Izzivi CQRS

- Več komponent → več **arhitekturne kompleksnosti**
- Potrebna je disciplina pri **usklajevanju modelov** (write ↔ read)
- **Sčasoma konsistentni** podatki
 - UI mora občasno tolerirati zamik.
- Več testiranja integracije
 - tokovi command → posodobitev read modela

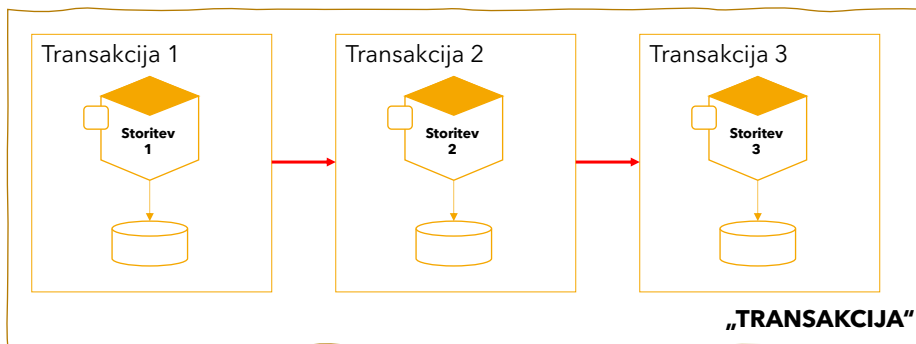
28

Zakaj potrebujemo vzorec Saga?

- V mikrostoritvah ima vsaka storitev **svoje podatke** → klasična **ACID transakcija** čez več baz ločenih storitev ni praktična.
- Poslovni procesi pa pogosto zahtevajo posodobitev **več storitev**
 - npr. naročilo + plačilo + zaloga
- **Saga** je vzorec za izvedbo porazdeljene transakcije kot **zaporedje lokalnih transakcij**.
- Konsistentnost se doseže **ščasoma** (*eventual consistency*), po potrebi s **kompensacijami**.

29

Kaj je vzorec Saga?



- **Saga** = niz korakov, kjer vsak korak:
 - izvede **lokalno transakcijo** v eni storitvi,
 - sproži **naslednji korak** (dogodek ali ukaz).
- Če korak ne uspe, Saga sproži **kompensacijske transakcije**, ki logično razveljavijo že izvedene korake.
- Pomembno: kompensacija ni "rollback" na nivoju baze, ampak **poslovno razveljavljanje** (npr. "rezervacijo zaloge" razveljavimo z "sprostitve rezervacije").

30

Dva načina izvedbe Sage

• Orkestracija (orchestrated saga)

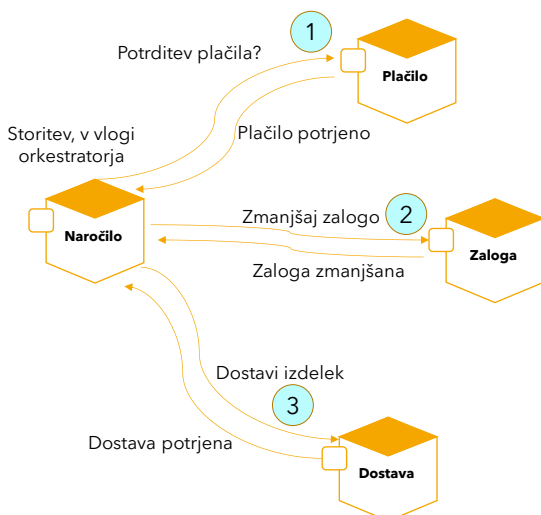
- Obstaja **orkestrator**, ki:
 - pošilja ukaze storitvam
 - spremlja odgovore/dogodke
 - odloča o naslednjih korakih in kompenzacijah
- **Prednosti:** jasen nadzor poteka in lažje razumevanje delovanja
- **Slabosti:** dodatna komponenta in tveganje "preveč centralne logike"

• Koreografija (choreographed saga)

- Ni centralnega vodje - storitve se odzivajo na **dogodke** in sprožajo naslednje dogodke.
- **Prednosti:** manj centralizacije - bolj šibko povezano.
- **Slabosti:** potek je težje sledljiv in večja potreba po dobrem opazovanju

31

Saga: Orkestracija



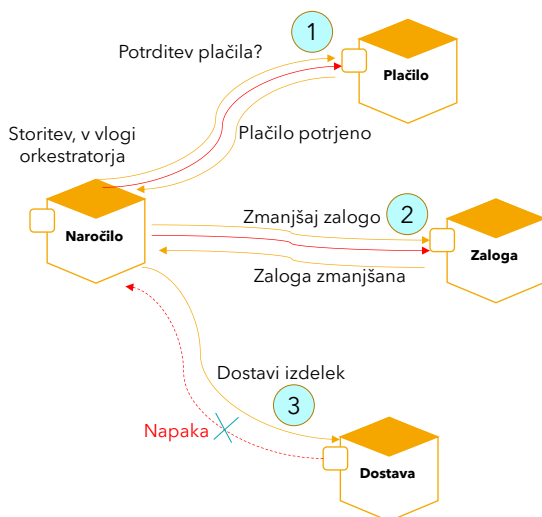
Storitev, v vlogi orkestratorja je storitev, ki kliče posamezne storitve v pravilnem zaporedju.

Na primer, če bi imeli spletno trgovino, bi po pritisku na gumb naročilo morali izvesti vrsto klicev na posamezne storitve, s čimer bi zagotovili dostavo izdelka stranki.

Klice ločenih storitev lahko izvedemo na osnovi sinhronih ali asinhronih vzorcev komunikacije.

32

Saga: Orkestracija

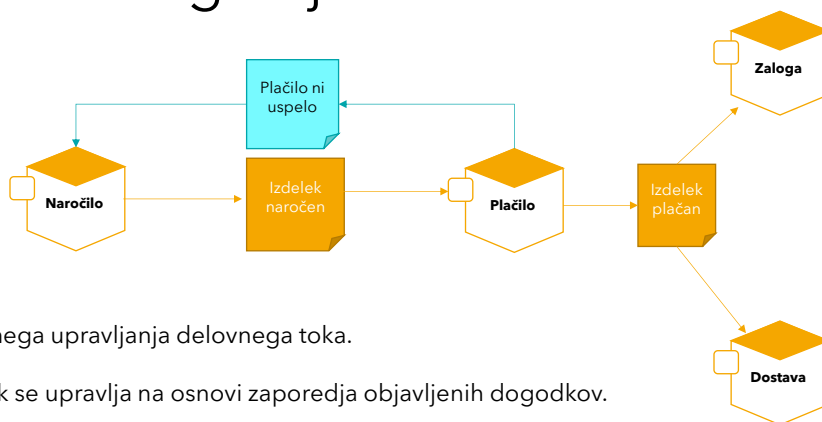


Pri orkestraciji storitev v vlogi orkestratorja čaka na odgovor prožene storitve.

Če pri proženju storitev pride do kakršnekoli napake, je orkestracijska storitev odgovorna za izvedbo klicev, s katerimi razveljavi stanje, do katerega je prišlo do trenutka napake.

33

Saga: Koreografija



Ni centralnega upravljanja delovnega toka.

Delovni tok se upravlja na osnovi zaporedja objavljenih dogodkov.

Ne čaka se na odgovor (kot v primeru orkestracije).

Na dogodke so prijavljeni odjemalci (storitve), ki se na dogodke ustrezno odzovejo.

34

Praktični izzivi Sage

Idempotentnost

- podvojena sporočila se morajo varno obdelati (isti ukaz/dogodek ne sme povzročiti dvojnega učinka).

Zanesljivost sporočil

- potreben je robusten mehanizem dostave (npr. broker).

Sledenje

- sledenje poslovnemu procesu čez storitve (correlation ID, tracing).

Kompensacije niso vedno popolne

- včasih je možna le "poslovna" razveljavitev, ne tehnična.

Timeouti in ročno posredovanje

- dolg proces lahko zahteva časovne omejitve in "manual override".

35

Potreba po programskih ogrodjih za razvoj mikrostoritev

- Konfiguracija
 - Omrežne lokacije zunanjih storitev, informacije, potrebne za avtentikacijo in avtorizacijo, itd.
 - Statična vs dinamična konfiguracija preko konfiguracijskega strežnika
- Zapisovanje dnevnikov
- Odkrivanje storitev
 - Registracija primerkov storitev
 - Pridobivanje informacij o omrežni lokaciji storitve
- Implementacija naprednih vzorcev (npr. vzorec prekinjevalca toka oz. Circuit Breaker)
- Preverjanje stanja storitev
- Izvajanje meritev z izbranimi metrikami
- ...

36

Ogradja za razvoj mikrostoritev

- V mikrostoritvenih rešitvah pogosto uporabimo ogradja, ki **out-of-the-box** podpirajo tipične potrebe:
 - konfiguracija po okoljih, health checks, metrike/telemetrija, odkrivanje storitev, odpornost (timeouts/circuit breaker), integracije z oblakom.
- Primeri ogrodij:
 - **Spring Boot**
 - **Spring Cloud** (config, discovery, routing, load balancing, circuit breakers ...)
 - **Dropwizard** (ops-friendly REST storitve, konfiguracija, metrike, logging ...)
 - **Quarkus** (Kubernetes-native, optimizirano za kontejnarje)
 - **Micronaut** (microservices + serverless)
 - **Helidon** (health/metrics/telemetrija/fault tolerance, integracije z observability orodji)
 - **Eclipse MicroProfile** (specifikacije za mikroservisno arhitekturo v Java svetu)
 - **ASP.NET Core** (HTTP API, gRPC, cloud-ready storitve)
 - **NestJS** (TypeScript/Node.js, modularna arhitektura za skalabilne storitve)
 - **FastAPI** (Python, visoko-zmogljivi API-ji)
 - **Go kit** (Go "toolkit" za mikroservise)